CST3492 - Java Enterprise Application Development
Academic Year 2026-2027 Semester 1

Programming Examination

Time Slot: 13:30-16:15, 21 October 2026
Venue: Lab 430, Jishi Building, Tongji University

Exam Rules

1. **You may use any AI tool during programming**: TRAE (with any model) is recommended, and other alternatives are also allowed. You may also look up documentation online.
2. **Do not communicate with anyone else in any form.** Everyone should work independently.
3. Part 3 is closed-book. Close your laptop before it starts.
4. Grading looks only at the files you submit and your closed-book answers. **You are not rewarded for writing more code or longer documents.** Features outside the scope earn no extra points.

Exam Schedule

Only the times in **bold** are fixed. During the programming time, plan your own pace; the suggested timing is a guide.

Time	Part	What you do
13:30–13:40	Setup	Unzip the starter project and run `./mvnw test` once under `part1-library` and `part2-review`
13:40–15:50	Programming Time	Part 1 and Part 2, in that order
(suggested 13:40–15:05)	Part 1: Build (60 points)	Spec and design first, commit; then implementation and tests, commit
(suggested 15:05–15:50)	Part 2: Review and Fix (25 points)	Decisions, tests and fixes, commit
15:50–16:00	Submission	Zip and upload to Canvas
16:00–16:15	Part 3: Closed-book Exam (15 points)	Close your laptop and write on the answer sheet

Part 1 · Build (60 points)

Requirement

> Library borrowing: the library lends books to two kinds of members, undergraduates and postgraduates. An undergraduate may hold at most 5 books at a time and keep a book for 14 days; a postgraduate may hold at most 10 books and keep a book for 28 days. The borrow day is day 0, and the last day of the loan period (day 14 or day 28) is the due date. Returning a book after the due date incurs a fine. The library expects to change its fine rule in future (for example, a discount during exam weeks), so switching to a different fine rule must not require changing the `Library` class.

Several things in this requirement are left open on purpose (for example, how much the fine is). There is no one to ask: **decide each open point yourself**. Write each decision as a numbered rule under "Rules" in your spec, and also list it under "Assumptions" with the reason and the rule number (see Step 2 below). You may discuss the open points with an AI, but the decisions are yours, and your spec, code and tests must agree with them.

Suggested Steps:

1. Read the requirement and list every point that is left open. Decide each one.
2. Fill in `docs/spec.md` from the template in the starter project. You may draft it with AI, but check every line yourself. Under **Rules**, write every rule as a numbered line (R1, R2, …) with concrete numbers: both the rules the requirement states and the rules you decided. Under **Assumptions**, list each open point, your decision, why, and the rule that states it. Under **Acceptance Criteria**, give at least one example per rule with a concrete input and expected result, including boundary values.
3. Fill in `docs/tasks.md`: break the work into small tasks, each with the files it touches and how to check it (a command or a test name).
4. Design the interfaces and classes by following good software design principles (see "Design Requirements" below), and write your design in `docs/design.md`.
5. **Git-commit** with the message "`spec`".
6. Ask the AI to implement your design and write tests. Check the expected values in the test cases against your spec yourself.
7. Run `./mvnw test` until everything passes. **Git-commit** with the message "`impl`".

How to write `docs/decisions.md`: Write this file yourself, in your own words, and update it after each important step instead of at the end. First record which AI tool and model you used. Then add one row for each key request you made to the AI (3–5 rows are enough; do not paste whole conversations): what you asked, in one line; what the AI produced; and what you accepted, changed or rejected, and why. For example: asked the AI to write tests from my acceptance criteria; it expected a fine of 50 cents on the due date; I changed it to 0, because my rule R2 says the due date is not overdue. If you accepted something unchanged, say so. An honest record scores better than a polished one.

Design Requirements

Your design must contain at least the following. Inside them, the structure is up to you.

Element	Requirement
`Member`	An **abstract class**, with two subclasses `Undergraduate` and `Postgraduate`. Each subclass has a constructor that takes the member ID, e.g. `new Undergraduate("S1")`. What differs between the two kinds of members belongs in the subclasses, not in hardcoded checks inside `Library`.
`FinePolicy`	An **interface** for the fine rule, with at least one implementation. `Library` depends only on the interface.
`LibraryException`	Your own exception class, with **at least two subclasses** for different kinds of rule violation (for example, the borrowing limit is reached).
Collections	Choose suitable collections and generic types for members, books and loans, and say why in `design.md`.

The starter project in `programming-exam/part1-library` already contains `Library.java` and an empty `Member.java`. `Library` is the entry point and must keep these public members:

```
public Library()   // uses your standard fine rule by default
public void addMember(Member member)
public void addBook(String isbn, int copies)
public void borrow(String memberId, String isbn, LocalDate date)
public int  returnBook(String memberId, String isbn, LocalDate date)   // calculates the value of
the fine, in cents
public int  borrowedCount(String memberId)
```

Conventions:

- Invalid input (unknown member or book, null argument, and so on) throws `IllegalArgumentException`. Valid input that breaks a borrowing rule throws a subclass of your `LibraryException`.

- Do not change the names, parameters or return types of the above methods, except for adding a `throws` clause for the `LibraryException`. You may add other constructors, methods and classes.
- Java 17 + JUnit 5. **No other dependencies.**
- Your tests may call `Library` and your other public classes, for example a `FinePolicy` implementation directly.

What must be contained in your submission

File	Content
`docs/spec.md`	Scope; rules, including the ones you decided; acceptance criteria (with concrete input values and expected results, including boundary values); and an Assumptions table linking each open point to its rule
`docs/tasks.md`	Task breakdown, each task stating how to check it
`docs/design.md`	Your classes and how they relate; why `Member` is an abstract class and `FinePolicy` an interface; your exception hierarchy; your choice of collections
`docs/decisions.md`	Which AI tool you used; your key requests, what the AI produced, what you accepted or changed (see "How to write `docs/decisions.md`" above). **Write this yourself.**
`src/main/java/...`	Your implementation
`src/test/java/...`	Your JUnit tests

How it is graded (60 points)

Part 1 is graded by examiners manually reading your files. We also run your own tests on your own code.

Item	Points	What we look for
Specification	15	Every open point is decided, written as a rule and listed under Assumptions; every rule has acceptance criteria with concrete values; boundaries are covered (for example the day before, on and after the due date; the 5th and 6th book for undergraduates)
Object-oriented design	15	The abstract class, interface and exception hierarchy are used for a real purpose; differences between members live in the subclasses; `design.md` explains the choices
Tests	15	Each rule has at least one test with exact expected values; boundary and exception cases are tested, with the specific exception type; tests agree with your spec
Implementation	10	The project builds, your tests pass, and the code does what your spec says
Development process	5	The `spec` commit comes before the `impl` commit, which comes before the `review` commit (only the order counts, not the clock time); `decisions.md` is an honest record

Part 2 · Review and Fix (25 points)

The project is in `programming-exam/part2-review`. Someone asked an AI to implement a fine ledger (`FineLedger`) from `docs/spec.md`, then asked a second AI session to review it. Read these three files first:

- `docs/spec.md` — the approved specification. It is the source of truth;
- `docs/ai-change.md` — background;
- `docs/ai-review-report.md` — the AI review report, with 8 findings (F1–F8).

Your task:

1. In `docs/review-decisions.md`, decide on each of F1–F8. Use exactly one of **Fix now / Verify first / Track later / Decline**, and give a one- or two-sentence reason (citing a spec rule or a line of code).
2. For every defect you confirm, **add a test to FineLedgerTest.java that fails against the original code**, then fix the defects by editing the `FineLedger.java` so that all tests pass.
3. The review report may have missed something. If you find a defect the report does not mention, write it in the last section of `review-decisions.md`, with a test and a fix.
4. **Git-commit** with the message "`review`".

How it is graded (25 points)

Item	Points
Your decisions on F1–F8	12
Added tests that expose real defects	6
Fixed code matches the specification	4
Finding the defect the report missed	3

Submission (15:50–16:00)

What git does here. The `programming-exam` folder is already a git repository. Each `git commit` saves a snapshot of your files **on your own computer**, in a hidden folder named `.git`. We read your commits from that folder to see the order of your work. Nothing here uses GitHub or any other website: you do not need an account, a remote repository or a network connection to commit. Modern IDEs such as TRAE, IntelliJ IDEA and VS Code commonly include git, so you can simply commit from their Git panel.

Before you zip:

1. Make sure you made three commits: `spec`, `impl`, `review`. Check with `git log --oneline` (you should see them, newest first, above the `starter` commit).
2. Do not delete or move the `.git` folder.

How to zip: Compress **the whole folder itself** (`programming-exam`), not the files you can see inside it. Reason: the `.git` folder is usually hidden: on macOS, Finder does not show it; on Windows, Explorer may not show it either. If you open the folder and select its visible contents to compress, the hidden `.git` folder will be left out and we cannot see your commits.

- macOS: in Finder, right-click the `programming-exam` folder, choose "Compress", then rename the file to `studentID.zip`. Or use the command on the last page.
- Windows: right-click the `programming-exam` folder, choose "Send to → Compressed (zipped) folder", then rename it to `studentID.zip`.

Upload `studentID.zip` to the programming exam's submission portal on Canvas.

Part 3 · Closed-book Exam (16:00–16:15, 15 points)

At 16:00, please close your laptop. The invigilator will hand out the answer sheet. All questions are about **what you yourself submitted**.

Appendix: Commands

```
# Run the tests (on Windows use: mvnw.cmd test)
cd programming-exam/part1-library
./mvnw test

# Commit (run inside the programming-exam folder)
git add -A
git commit -m "spec"        # then "impl", then "review"
git log --oneline           # check your commits

# Zip (macOS / Linux; run in the folder that contains programming-exam)
zip -r studentID.zip programming-exam
```

If git misbehaves, please raise your hand. Never delete or edit the `.git` folder under any circumstance.